



## **Compiler Construction Lab 5**

**Name: Kashaf Ahmad**

**Sap ID: 53155**

**Submitted To: Ma`am Hasnat Bibi**

## Task 1:

```
%{
#include <stdio.h>

int keywords = 0;
int identifiers = 0;
int operators = 0;
int numbers = 0;

}%

%%

"int"|"float"|"char"|"double"|"if"|"else"|"while"|"for"|"return"|"void" { keywords++; }
[a-zA-Z_][a-zA-Z0-9_]* { identifiers++; }
"+"|"-"|"*"|" "/"|"%"|"="|"=="|"!="|"<"|>"|<="|>=" { operators++; }
[0-9]+(\.[0-9]+)? { numbers++; }
[ \t\n] ;
. ;

%%

int main(){
    yylex();
    printf("Keywords: %d\n", keywords);
    printf("Identifiers: %d\n", identifiers);
    printf("Operators: %d\n", operators);
    printf("Numeric literals: %d\n", numbers);
    return 0;
}

int yywrap()
{
    return 1;
}
```

```

student@proglab2-44:~$ flex task1.l
student@proglab2-44:~$ gcc lex.yy.c -o task1
student@proglab2-44:~$ ./task1
int a = 5;
float b = 3.14;
a = a+b;Keywords: 2
Identifiers: 5
Operators: 4
Numeric literals: 2

```

## Task 2:

```

%{
#include <stdio.h>

int line = 1;

%}

%%

"/".*      ;

"/"([^\]|\"+[/])*\\"+ "/"    ;

\n    { line++; printf("\nLine %d: ", line); }

.    { printf("%s", yytext); }

%%

int main()
{
    printf("Line 1: ");
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}

```

```
student@proglab2-44:~$ nano task2.1
student@proglab2-44:~$ flex task2.1
student@proglab2-44:~$ gcc lex.yy.c -o task2
student@proglab2-44:~$ ./task2
Line 1: int a = 5;
int a = 5;
Line 2: float b = 3.14;
float b = 3.14;
Line 3: printf("Hello");
printf("Hello");
```

## Task 3:

```
%{
#include <stdio.h>
%}
%%

"+" { printf("Arithmetic Operator: +\n"); }
"-" { printf("Arithmetic Operator: -\n"); }
"*" { printf("Arithmetic Operator: *\n"); }
"/" { printf("Arithmetic Operator: /\n"); }
"%" { printf("Arithmetic Operator: %%\n"); }
"<" { printf("Relational Operator: <\n"); }
">" { printf("Relational Operator: >\n"); }
"<=" { printf("Relational Operator: <=\n"); }
">=" { printf("Relational Operator: >=\n"); }
"==" { printf("Relational Operator: ==\n"); }
"!=" { printf("Relational Operator: !=\n"); }
"=" { printf("Assignment Operator: =\n"); }
"+=" { printf("Assignment Operator: +=\n"); }
"-= " { printf("Assignment Operator: -=\n"); }
"*=" { printf("Assignment Operator: *=\n"); }
"/=" { printf("Assignment Operator: /=\n"); }
```

```

"%=" { printf("Assignment Operator: %%=\\n"); }
"++" { printf("Assignment Operator: ++\\n"); }
"--" { printf("Assignment Operator: --\\n"); }
[ \\t\\n];
.;
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}

```

```

student@proglab2-44:~$ nano task3.1
student@proglab2-44:~$ flex task3.1
student@proglab2-44:~$ gcc lex.yy.c -o task3
student@proglab2-44:~$ ./task3
a + b
Arithmetic Operator: +
a >= b
Relational Operator: >=
a += 5
Assignment Operator: +=
c
c++
Assignment Operator: ++

```